



# Desafio C++: Sistema de Inventário de RPG

Você está desenvolvendo o sistema de inventário para um novo jogo de RPG de fantasia. O jogador pode carregar diferentes tipos de itens em sua mochila.

Novamente, o foco é usar **POO por valor** com `std::variant` e `std::visit`, evitando alocação dinâmica ( `new` , ponteiros inteligentes) e funções virtuais.

## Suas Tarefas

### 1. Criar as Classes de Itens (POO)

Crie três classes que representam os itens. Pense no encapsulamento (atributos privados e métodos para modificá-los ou lê-los).

- Arma
  - Estado: `std::string nome` , `int dano` , `int durabilidade` (padrão: 100).
  - Comportamento:
    - Construtor para inicializar nome e dano.
    - `getNome()` , `getDano()` , `getDurabilidade()` .
    - `atacar()` : Reduz a durabilidade em 10. Se a durabilidade já estiver em 0, não faz nada.
    - `reparar(int quantidade)` : Aumenta a durabilidade, mas com limite máximo de 100.
    - `melhorar()` : Aumenta o dano em +5.
- Pocao
  - Estado: `std::string tipo` (ex: "Vida", "Mana"), `int poder` , `bool consumida` (padrão: false).
  - Comportamento:
    - Construtor para inicializar tipo e poder.
    - Getters necessários.
    - `beber()` : Retorna o `poder` se não estiver consumida e muda `consumida` para `true`. Se já estiver consumida, retorna 0.
    - `aprimorar()` : Dobra o `poder` (só funciona se não estiver consumida).
- Pergaminho
  - Estado: `std::string feitiço` , `bool lido` (padrão: false).
  - Comportamento:
    - Construtor para inicializar o feitiço.
    - Getters necessários.
    - `ler()` : Marca como `lido = true` .
    - `restaurarMagia()` : Retorna o pergaminho para o estado `lido = false` (tornando-o utilizável novamente).

### 2. Definir o Variant

- Crie o alias `Item` usando `std::variant` para agrupar `Arma`, `Pocao` e `Pergaminho`.

### 3. Criar a Classe Gerenciadora ( Inventario )

Esta classe representa a mochila do jogador.

- Atributo: `std::vector<Item> mochila`.
- Método: `adicionarItem(Item item)`.
- Método: `inspecionarMochila()`: Usa `std::visit` para mostrar os detalhes de **cada** item.
  - Exemplo para **Arma**: "[Arma] Espada Longa - Dano: 15, Durabilidade: 90/100"
- Método: `receberBencaoDosDeuses()`: O jogador encontrou um altar! Você deve percorrer todos os itens e, usando `std::visit` e referências ( `auto&` ), aplicar a seguinte lógica de melhoria mágica:
  - Se for **Arma**: chama `reparar(100)` (restaura tudo) e chama `melhorar()`.
  - Se for **Pocao**: chama `aprimorar()`.
  - Se for **Pergaminho**: chama `restaurarMagia()`.

### Dica de Ouro

No método `receberBencaoDosDeuses()`, preste atenção para garantir que a lambda ou struct passada para o `std::visit` receba o item **por referência** ( `auto& item` ), senão você estará modificando apenas uma cópia temporária e a mochila real continuará igual!